AI2002
Artificial Intelligence
Lecture 15

Mahzaib Younas

Lecturer, Department of Computer Science

FAST NUCES CFD

Connection Between ∀ *and* ∃

- Asserting that **"Everyone dislikes parsnips"** is the same as asserting there does not exist someone who likes them, and vice versa:

$$\forall x \ \neg Likes(x, Parsnips) \quad \text{is equivalent to} \quad \neg\exists x \ Likes(x, Parsnips)$$

- We can go one step further: **"Everyone likes ice cream"** means that there is no one who does not like ice cream:

$$\forall x \ Likes(x, IceCream) \quad \text{is equivalent to} \quad \neg\exists x \ \neg Likes(x, IceCream)$$

- **∀ is really conjunction** over the universe of objects while **∃ is a disjunction**.
- **Quantifiers obey De Morgan's rules**. The De Morgan rules for quantified and unquantified sentences are as follows:

$\forall x \ \neg Likes(x, Parsnips)$ is equivalent to $\neg\exists x \ Likes(x, Parsnips)$

$\forall x \ Likes(x, IceCream)$ is equivalent to $\neg\exists x \ \neg Likes(x, IceCream)$

$$\neg\exists x \ P \equiv \forall x \ \neg P \qquad \neg(P \vee Q) \equiv \neg P \wedge \neg Q$$
$$\neg\forall x \ P \equiv \exists x \ \neg P \qquad \neg(P \wedge Q) \equiv \neg P \vee \neg Q$$
$$\neg\exists x \ \neg P \equiv \forall x \ P \qquad P \wedge Q \equiv \neg(\neg P \vee \neg Q)$$
$$\neg\forall x \ \neg P \equiv \exists x \ P \qquad P \vee Q \equiv \neg(\neg P \wedge \neg Q)$$

Equality

- We can use the equality symbol to signify *that two* ***terms refer to the same object***.
- Example:

$$Father(John) = Henry$$

- The equality symbol can be used to state facts about a given function.
- To say that ***Richard has at least two brothers***,

$$\exists x, y \;\; Brother(x, Richard) \wedge Brother(y, Richard)$$

- The above sentence does not have the intended meaning. The correct version is:

$$\exists x, y \;\; Brother(x, Richard) \wedge Brother(y, Richard) \wedge \neg(x = y)$$

FOL Syntax

$$
\begin{aligned}
\textit{Sentence} &\rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence} \\
\textit{AtomicSentence} &\rightarrow \textit{Predicate} \mid \textit{Predicate}(\textit{Term}, \ldots) \mid \textit{Term} = \textit{Term} \\
\textit{ComplexSentence} &\rightarrow (\ \textit{Sentence}\ ) \mid [\ \textit{Sentence}\ ] \\
&\mid \neg\ \textit{Sentence} \\
&\mid \textit{Sentence} \wedge \textit{Sentence} \\
&\mid \textit{Sentence} \vee \textit{Sentence} \\
&\mid \textit{Sentence} \Rightarrow \textit{Sentence} \\
&\mid \textit{Sentence} \Leftrightarrow \textit{Sentence} \\
&\mid \textit{Quantifier Variable}, \ldots \textit{Sentence}
\end{aligned}
$$

FOL Syntax

$$
\begin{aligned}
\mathit{Term} &\rightarrow \mathit{Function}(\mathit{Term}, \ldots) \\
&\mid \mathit{Constant} \\
&\mid \mathit{Variable} \\
\\
\mathit{Quantifier} &\rightarrow \forall \mid \exists \\
\mathit{Constant} &\rightarrow A \mid X_1 \mid \mathit{John} \mid \cdots \\
\mathit{Variable} &\rightarrow a \mid x \mid s \mid \cdots \\
\mathit{Predicate} &\rightarrow \mathit{True} \mid \mathit{False} \mid \mathit{After} \mid \mathit{Loves} \mid \mathit{Raining} \mid \cdots \\
\mathit{Function} &\rightarrow \mathit{Mother} \mid \mathit{LeftLeg} \mid \cdots \\
\text{OPERATOR PRECEDENCE} &: \neg, =, \wedge, \vee, \Rightarrow, \Leftrightarrow
\end{aligned}
$$

Assertions and Queries in FOL

- Assertion
  - used to add the data in KB
  - TELL is used for assertion

- Queries
  - to retrieve the data from KB
  - Ask is used for Queries

Assertions

- Sentences are added to a knowledge base using TELL are called assertions
- We want to TELL things to the KB

  TELL(KB, King(John))

  TELL(KB, ∀ x king(x) => Person(x))

  John is a king and that king is a person.

Queries

- Questions are asked to the knowledge base using ASK called as queries or goals.
- We want to ASK things to the KB

$$\textsc{Ask}(KB,\ Person(John))$$

Inferences in First Order Logics

- Two ways of inference in first order logic
- The *first-order* inference can be done by converting the knowledge base to *propositional* logic
  - some simple inference rules that can be applied to sentences with quantifiers to obtain sentences without quantifiers.

- The inference methods that manipulate first-order sentences directly.

Inference Rule for Quantifiers

- **Universal Instantiation**
  - We can **infer** **any sentence** obtained by *substituting a ground term for the variable.*
  - Ground term is the term that is without variables.

E.g., $\forall x \;\; King(x) \wedge Greedy(x) \Rightarrow Evil(x)$ yields

$$King(John) \wedge Greedy(John) \Rightarrow Evil(John)$$
$$King(Richard) \wedge Greedy(Richard) \Rightarrow Evil(Richard)$$
$$King(Father(John)) \wedge Greedy(Father(John)) \Rightarrow Evil(Father(John))$$
$$\vdots$$

Universal Instantiation

$$\frac{\forall v \ \alpha}{\text{SUBST}(\{v/g\}, \alpha)}$$

for any variable v and ground term g

- SUBST(θ, α) denotes the result of applying the substitution θ to the sentence α.

Universal Instantiation Example

E.g., $\forall x \;\; King(x) \wedge Greedy(x) \;\Rightarrow\; Evil(x)$ yields

$$King(John) \wedge Greedy(John) \;\Rightarrow\; Evil(John)$$
$$King(Richard) \wedge Greedy(Richard) \;\Rightarrow\; Evil(Richard)$$
$$King(Father(John)) \wedge Greedy(Father(John)) \;\Rightarrow\; Evil(Father(John))$$
$$\vdots$$

The three sentences are obtained with substitutions
x/John}, {x/Richard }, and **{x/Father (John)}**.

Existential Quantifiers

- The variable is replaced by a single ***new constant symbol***.
- For any sentence ***α***, variable ***v***, and constant symbol ***k*** that ***does not appear elsewhere in the knowledge base***,

$$\frac{\exists v \;\; \alpha}{\text{SUBST}(\{v/k\}, \alpha)}$$

- C_1 is the constant which does not appear elsewhere in the knowledge base. Such a constant is called **Skolem constant** and the process is called **Skolemization.**

E.g., $\exists x \;\; Crown(x) \wedge OnHead(x, John)$ yields

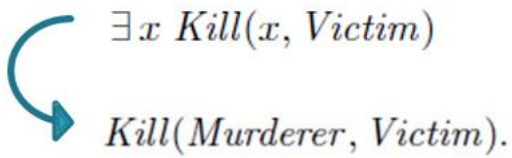
$$Crown(C_1) \wedge OnHead(C_1, John)$$

Inference Rules for Quantifiers

- **Universal Instantiation** can be applied **several** times to add new sentences; the new KB is **logically equivalent** to the old one.

- **Existential Instantiation** can be applied **once** to replace the existential sentence; the new KB is **NOT equivalent** to the old,

- But it is **inferentially equivalent** in a sense that it is *satisfiable iff the old KB was satisfiable*.



Example:

- Suppose the **KB** consists of following sentences:

$$\forall x \;\; King(x) \wedge Greedy(x) \Rightarrow Evil(x)$$
$$King(John)$$
$$Greedy(John)$$
$$Brother(Richard, John)$$

Example

- Instantiating the universal sentence in ***all*** possible ways, we have

$$King(John) \wedge Greedy(John) \Rightarrow Evil(John)$$
$$King(Richard) \wedge Greedy(Richard) \Rightarrow Evil(Richard)$$
$$King(John)$$
$$Greedy(John)$$
$$Brother(Richard, John)$$

This KB is propositionalized

Problem in Propositionalizing

- Propositionalizing seems to generate the lots of **irrelevant sentences**.

For Example

- Given the query **Evil(x)** for the following KB,

$$King(John) \wedge Greedy(John) \Rightarrow Evil(John)$$
$$King(Richard) \wedge Greedy(Richard) \Rightarrow Evil(Richard)$$
$$King(John)$$
$$Greedy(John)$$
$$Brother(Richard, John)$$

It may generate sentences such as

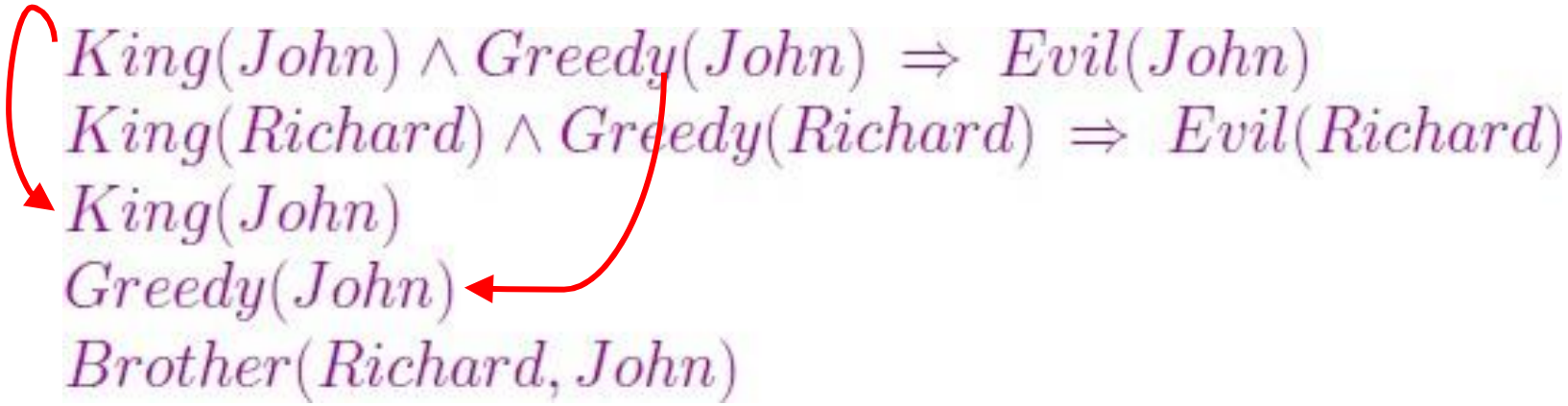
King(Richard) ∧ Greedy(Richard) ⇒ Evil(Richard).

The inference is that John is evil

Solution:

- **The inference is that John is evil—{x/John}** solves the query **Evil(x)**—The **substitution θ ={x/John}** achieves that goal.
- If there is some substitution ***θ***
  - that makes **the premise of the implication** identical to
  - sentences *already in the knowledge base*,
  - then we can assert the conclusion of the implication, after
  - applying θ.
- In this case, the substitution $\theta = \{x/John\}$ achieves that aim.

Problem



makes **the premise of the implication** identical to sentences ***already in the knowledge base***

Solution

- Suppose that instead of knowing **Greedy(John)**, we know that ***everyone is greedy:***

$$\forall x \;\; King(x) \wedge Greedy(x) \Rightarrow Evil(x)$$
$$King(John)$$
$$\sout{Greedy(John)}$$
$$Brother(Richard, John)$$

- Then we would still able to conclude that **Evil(John)**, because we know that
  - **John is a king (given)**
  - **John is greedy (because everyone is greedy).**

Unification

- The **Unify algorithm** takes two sentences and returns a **unifier** for them if one exists:

$$\text{UNIFY}(p, q) = \theta \text{ where } \text{SUBST}(\theta, p) = \text{SUBST}(\theta, q) .$$

- Suppose we have a query **AskVars(Knows(John, x)): whom does John know?**
- To answer this question, we have to find all sentences in the knowledge base that unify with **Knows(John, x).**

Unification

The condition for unification are

1. Predicate symbol must be same in both sentences.
2. Number of arguments in both expression must be identical
3. Unification will fail if the two similar variable present in the same expression.

Example

$$\text{UNIFY}(Knows(John, x),\ Knows(John, Jane)) = \{x/Jane\}$$
$$\text{UNIFY}(Knows(John, x),\ Knows(y, Bill)) = \{x/Bill, y/John\}$$
$$\text{UNIFY}(Knows(John, x),\ Knows(y, Mother(y))) = \{y/John, x/Mother(John)\}$$
$$\text{UNIFY}(Knows(John, x),\ Knows(x, Elizabeth)) = fail\ .$$

- The last unification fails because **x** cannot take on the values **John** and **Elizabeth** at the same time
- **Knows(x, Elizabeth)** means **"Everyone knows Elizabeth,"** and we can infer that **John knows Elizabeth**

Example

- This problem arises because two sentences happen to use the **same variable** name, **x**
- The problem can be avoided by **standardizing apart** which means **renaming its variables** to avoid name clashes.
- **Standardizing apart** eliminates overlap of variables.
- For example, we can rename **x** in **Knows(x, Elizabeth)** to $\mathbf{x_{17}}$ (a new variable name) without changing its meaning.

$$\text{UNIFY}(Knows(John, x),\ Knows(x_{17}, Elizabeth)) = \{x/Elizabeth, x_{17}/John\}$$

Example:

1. O (F (y), y), O (F(x), J)
2. Q(y, G(A, B)), Q(G(x, x), y)

Example 1 Solution:

- ***Progressive unification:***

- O (F (y), y), O (F(x), J) : {} *needs recursion*
- O (F (y), y), O (F(x), J) : {y/x}
- O (F (x), x), O (F (x), J) : {y/x, x/J} = {y/J, x/J}
- O (F (J), J), O (F (J), J) : {y/x, x/J} = {y/J, x/J}

Example 2 Solution

- ***Progressive unification:***

- Q(y, G(A, B)), Q(G(x, x), y) : {y/G(x, x)},
- Q(G(x, x), G(A, B)), Q(G(x, x), G(x, x)) : {y/G(x, x)} *needs recursion*
- Q(G(x, x), G(A, B)), Q(G(x, x), G(x, x)) : {y/G(x, x), x/A}
- Q(G(A, A), G(A, B)), Q(G(A, A), G(A, A)) : {y/G(x, x), x/A}
- **Cannot unify *constant A* with *constant B*.**

Resolution

Here's the rule for first-order resolution.

$$\frac{\alpha \vee \varphi \qquad \neg \psi \vee \beta}{(\alpha \vee \beta)\theta} \qquad MGU(\varphi, \psi) = \theta$$

Example 1:

$$\frac{P(x) \vee Q(x,y) \qquad \neg P(A) \vee R(B,z)}{(Q(x,y) \vee R(B,z))\theta}$$

$$Q(A,y) \vee R(B,z)$$

$$\theta = \{x/A\}$$

Resolution

Example 2: In this example, there is ***x*** **in the two sentences** which are actually *different*.

$$P(x) \vee Q(x,y)$$

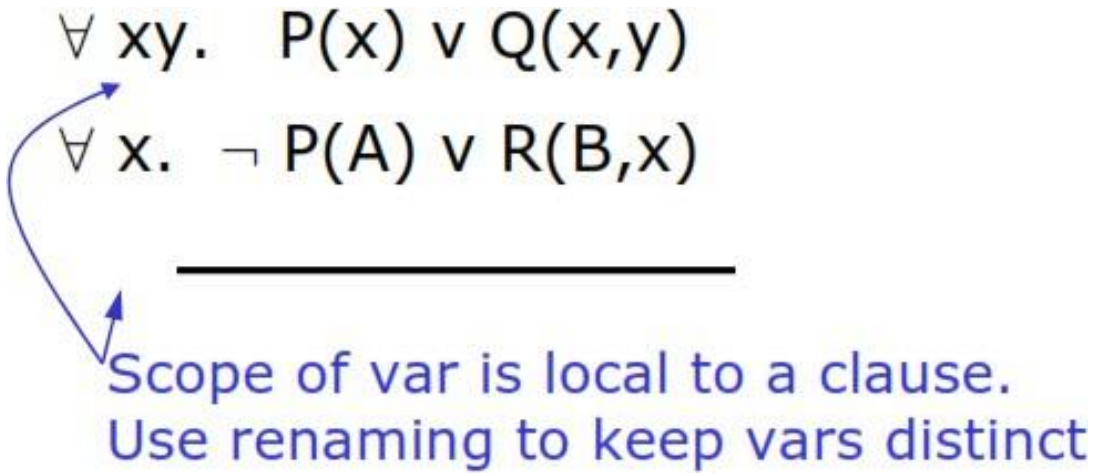
$$\neg P(A) \vee R(B,x)$$

Furthermore, ***there is an implicit universal quantifier*** on the outside of each of these sentences.

$$\forall xy. \quad P(x) \vee Q(x,y)$$

$$\forall x. \quad \neg P(A) \vee R(B,x)$$

Resolution



Rename the variables (Standardizing apart) in the two sentences so that they don't share any variables in common. **Standardizing apart** eliminates overlap of variables.

$$\forall x_1 y.\quad P(x_1) \vee Q(x_1, y)$$
$$\forall x_2.\quad \neg P(A) \vee R(B, x_2)$$

Resolution

$\forall x_1 y. \quad P(x_1) \vee Q(x_1, y)$

$\forall x_2. \quad \neg P(A) \vee R(B, x_2)$

$$\frac{\forall x_1 y. \quad P(x_1) \vee Q(x_1, y) \qquad \forall x_2. \quad \neg P(A) \vee R(B, x_2)}{(Q(x_1, y) \vee R(B, x_2))\theta}$$

$$\theta = \{x_1 / A\}$$

$\forall x_1 y. \quad P(x_1) \vee Q(x_1, y)$

$\forall x_2. \quad \neg P(A) \vee R(B, x_2)$

$$\frac{}{(Q(x_1, y) \vee R(B, x_2))\theta}$$

$Q(A, y) \vee R(B, x_2)$

Factoring

- **Factoring**—the removal of redundant literals—to the first-order case.
- Propositional factoring reduces two literals to one if they are *identical*;
- First-order factoring reduces two literals to one if they are *unifiable*.
- The unifier must be applied to the entire clause.
- *The combination of binary resolution and factoring is complete*

Factoring

In factoring,

- **unify two literals** within a **single** clause,
  - **alpha** and **beta** in this case, with **unifier theta**,
- **drop one of them** from the clause (it doesn't matter which one),
- apply the unifier to the whole clause.

$$\frac{\alpha \vee \beta \vee \gamma}{(\alpha \vee \gamma)\theta} \qquad \theta = \mathrm{MGU}(\alpha, \beta)$$

Binary Resolution, combined with factoring, is **complete**.

Factoring

$$Q(y) \vee P(x,y) \vee P(v,A)$$

- we can apply factoring to this sentence, by unifying $\boldsymbol{P(x,y)}$ and $\boldsymbol{P(v,A)}$.

$$\frac{Q(y) \vee P(x,y) \vee P(v,A)}{(Q(y) \vee P(x,y))\{x/v, y/A\}}$$
$$Q(A) \vee P(v,A)$$

Converting to Clausal Form

Eliminate $\rightarrow, \leftrightarrow$

$$\alpha \rightarrow \beta \qquad \neg\alpha \vee \beta$$

Drive in $\neg$

$$\neg(\alpha \vee \beta) \qquad \neg\alpha \wedge \neg\beta$$

$$\neg(\alpha \wedge \beta) \qquad \neg\alpha \vee \neg\beta$$

$$\neg\neg\alpha \qquad \alpha$$

$$\neg\exists x.\ P(x) \qquad \forall x. \neg P(x)$$

$$\neg\forall x. P(x) \qquad \exists x. \neg P(x)$$

Converting to Clausal Form

- Rename variables apart

$$\forall x.\, \exists y.\, (P(x) \rightarrow \forall x.\, Q(x, y))$$

$$\forall x_1.\, \exists y_2.\, (P(x_1) \rightarrow \forall x_3.\, Q(x_3, y_2))$$

- Skolemization
  - ***Skolem Constant:*** Substitute brand new name for each *existentially quantified variable*
  - $\exists\, x.\, P(x)$ $\qquad$ $P(Fred)$
  - $\exists\, x.\, P(x, y)$ $\qquad$ $P(X11, Y13)$
  - $\exists\, x.\, P(x) \wedge\, Q(x)$ $\qquad$ $P(Blue)\ \wedge\ Q(Blue)$

Converting to Clausal Form

Skolemization

Skolem Function: Substitute a **new function** of all universally quantified variables in enclosing scopes for each *existentially quantified variable*.

"There is someone who is loved by everyone"

$$\exists\, y.\, \forall\, x.\, Loves\,(x, y)$$

$$\forall\, x.\, Loves(x, Englebert)$$

"Everybody loves somebody"

$$\forall\, x.\, \exists\, y.\, Loves\,(x, y)$$

$$\forall\, x.\, Loves(x, Beloved(x))$$

Converting in Casual Form

- Prenex Form
- Drop universal quantifiers
- Convert to CNF
- Rename the variables in each clause, if necessary

Steps to Convert in resolution

1. **Negate the conclusion statement**
2. **Convert all the given statement/facts in FOL**
3. **Convert FOL in CNF**
4. **Draw the resolution Graoh**

Solution: Step 1

- First we create the predicate of conclusion

- Ali play Cricket

Play (Ali, Cricket)

Example:

1. Ali likes all kind of games.
2. Cricket and football are games.
3. Anything anyone play and is not killed is game.
4. Ahmad play cricket and still alive.
5. Abdullah play that ahmad play.

- Prove
- Ali Play Cricket.

Step 3: Convert FOL to CNF

Ali likes all kind of games.

$\forall x\ game(x) \rightarrow likes(ali, x)$

$\sim$game(x) $\vee$ likes(ali,x)

Cricket and football are games.

Game(football) Game(Cricket)

Anything anyone play and is not killed is game.

$\forall x\ \forall y\ play(x, y) \wedge \sim killed(x) \rightarrow$ game(y)

$\sim$play(x,y) $\vee$ killed(x) $\vee$ game(y)

Ahmad play cricket and still alive.

Play(ahmad, cricket) $\wedge$ alive(Ahmad)

Abdullah play that ahmad play.

$\forall x\ killed(x) \rightarrow \sim alive(x)$

$\forall x\ alive(x) \rightarrow \sim killed(x)$

Step 4: Resolution Graph

Given Premises	**Resolution**
~game(x) ∨ likes(ali,x)	1
Game(football) Game(Cricket)	2
~play(x,y) ∨ killed(x) ∨ game(y)	3
Play(ahmad, cricket) ∧ alive(Ahmad)	4
$\sim killed\ x \vee \sim alive\ x$	5
$\sim alive\ x \vee \sim killed(x)$	6
~Like (Ali, Cricket)	Conclusion

Resolution Graph

Given Premises	**Substitution**	**Resolution**
~game(x) ∨ likes(ali,x)		1
Game(football) Game(Cricket)		2
~play(x,y) ∨ killed(x) ∨ game(y)		3
Play(ahmad, cricket) ∧ alive(Ahmad)		4
$\sim killed\ x \vee \sim alive\ x$		5
$\sim alive\ x \vee \sim killed(x$		6
~Like (Ali, Cricket)		Conclus
~game(Cricket) ∨ likes(ali, Cricket) ∨ ~Like (Ali, Cricket)	*x/cricket*	~game(Cricket)

Resolution Graph

Given Premises	Substitution	Resolution
~game(x) ∨ likes(ali,x)		1
Game(football) Game(Cricket)		2
~play(x,y) ∨ killed(x) ∨ game(y)		3
Play(ahmad, cricket) / alive(Ahmad)		4
$\sim killed(x) \vee \sim alive(x)$		5
$\sim alive(x) \vee \sim killed(x)$		6
~Like (Ali, Cricket)		Conclusion
~game(Cricket) ∨ likes(ali, Cricket) ∨ ~Like (Ali, Cricket)	*x/cricket*	~game(Cricket)
~play(x,Cricket) ∨ killed(x) ∨ game(Cricket) ∨ ~game(Cricket)	*y/cricket*	~play(x,Cricket) ∨ killed(x)

Given Premises	Substitution	Resolution
~game(x) ∨ likes(ali,x)		1
Game(football) Game(Cricket)		2
~play(x,y) ∨ killed(x) ∨ game(y)		3
Play(ahmad, cricket) / alive(Ahmad)		4
$\sim killed(x) \vee \sim alive(x)$		5
$\sim alive(x) \vee \sim killed(x)$		6
~Like (Ali, Cricket)		Conclusion
~game(Cricket) ∨ likes(ali, Cricket) ∨ ~Like (Ali, Cricket)	*x/cricket*	~game(Cricket)
~play(x,Cricket) ∨ killed(x) ∨ game(Cricket) ∨ ~game(Cricket)	*y/cricket*	~play(x,Cricket) ∨ killed(x)
Play(ahmad, cricket)∨ ~play(ahmad,Cricket) ∨ killed(ahmad)	*x/ahmad*	Killed(ahmad)

Resolution Graph

Given Premises	Substitution	Resolution
~game(x) ∨ likes(ali,x)		1
Game(football) Game(Cricket)		2
~play(x,y) ∨ killed(x) ∨ game(y)		3
Play(ahmad, cricket) / alive(Ahmad)		4
$\sim killed(x) \vee \sim alive(x)$		5
$\sim alive(x) \vee \sim killed(x)$		6
~Like (Ali, Cricket)		Conclusion
~game(Cricket) ∨ likes(ali, Cricket) ∨ ~Like (Ali, Cricket)	*x/cricket*	~game(Cricket)
~play(x,Cricket) ∨ killed(x) ∨ game(Cricket) ∨ ~game(Cricket)	*y/cricket*	~play(x,Cricket) ∨ killed(x)
Play(ahmad, cricket)∨ ~play(ahmad,Cricket) ∨ killed(ahmad)	*x/ahmad*	Killed(ahmad)
$\sim killed(ahmad) \vee \sim alive(ahmad)$∨ Killed(ahmad)	*x/ahmad*	$\sim alive(ahmad)$

Example 2:

A knowledge base has the following statements:

- If there is gas in the tank and the fuel line is okay, then there is gas in the engine;
- If there is gas in the engine and a good spark, the engine runs;
- If there is power to the plugs and the plugs are clean, a good spark is produced;
- If the battery is charged and the cables are okay, then there is power to the plugs.

Convert these statements into FOL, Convert each FOL statement in (a) to CNF, Using resolution, prove that engine runs.

Example 2:

- GasInTank ∧ FuelLineOK → GasInEngine - True (given)
- ChargedBattery ∧ CablesOkay → PowerToPlugs - True (given)
- PowerToPlugs ∧ CleanPlugs →** GoodSpark** - True (given)
- GasInEngine ∧ GoodSpark → EngineRuns - True

- **¬GasTank(x) ∨ ¬FuelLineOkay(x) ∨ GasEngine(x)**
- **¬GasEngine(x) ∨ ¬GoodSpark(x) ∨ EngineRuns(x)**
- **¬PowerPlugs(x) ∨ ¬CleanPlugs(x) ∨ GoodSpark(x)**
- **¬BatteryCharged(x) ∨ ¬CablesOkay(x) ∨ PowerPlugs(x)**
- **Facts in CNF:**
  - **BatteryCharged(Car)**
  - **CablesOkay(Car)**
  - **GasTank(Car)**
  - **FuelLineOkay(Car)**
  - **CleanPlugs(Car)**

Green Trick

- We **can ask for an answer** to a question with resolution.
- If the desired conclusion is that there exists an $\boldsymbol{x}$ such that $\boldsymbol{P(x)}$, we'll figure out what value of $\boldsymbol{x}$ makes $\boldsymbol{P(x)}$ true.
- **Green's trick,** named after **Cordell Green**, who pioneered the use of logic.
- "There exists a sequence of actions such that, if I do them in my initial state, my goal will be true at the end."

Green Trick

All men are mortal and Socrates is a man.

- We want to know whether there are *any mortal available in the knowledge base*.
- Use resolution to get answers.

- The desired conclusion, negated and turned into clausal form would be

$$\neg Mortal\ (x)$$

Green's trick will be to add a special extra literal onto that clause, of the form $Answer(x)$**.**

Green Trick

1.	$\neg \text{Man}(x) \vee \text{Mortal}(x)$	
2.	$\text{Man}(\textit{Socrates})$	
3.	$\neg \text{Mortal}(x) \vee \text{Answer}(x)$	
4.		
5.		

Green Trick

Example:

All men are mortal and that Socrates is a man.

1.	$\neg$Man(x) $\vee$ Mortal(x)	
2.	Man(*Socrates*)	
3.	$\neg$Mortal(x) $\vee$ Answer(x)	
4.	Mortal(*Socrates*)	1,2
5.	Answer(*Socrates*)	**3,4**

We can resolve lines (3 and 4), substituting *Socrates* for x, and get *Answer*.